

LLMGuard: An Integrity Attestation Framework for Large Language Models

Aayush Rai
Department of Computer Science
Kansas State University
Manhattan, Kansas, USA
aayushrai@ksu.edu

Eugene Vasserman (Advisor)
Department of Computer Science
Kansas State University
Manhattan, Kansas, USA
eyv@ksu.edu

Abstract—Large language models deployed in production systems are increasingly attractive targets for tampering: an attacker who compromises the host process can modify the model at several points in its lifecycle — on disk before load, in memory after load, or at inference time — and the user downstream has no way to detect that the model is no longer the model that was intended to be deployed. Existing defenses tend to address one stage of the lifecycle. Hardware-rooted file integrity tools such as the Trusted Platform Module combined with the Linux Integrity Measurement Architecture cover load-time tampering but stop tracking the model once it is in process memory. Behavioral attestation in the AttestLLM family covers runtime tampering by checking the live model against secret triggers held in a Trusted Execution Environment, but does not provide load-time guarantees. This paper presents LLMGuard, an integrity attestation framework that combines the two approaches into a layered design. The threat model assumes a userspace adversary who fully owns the LLM process; the framework provides load-time integrity through TPM+IMA and runtime integrity through TEE-protected behavioral attestation, with the two layers closing complementary gaps in the model’s lifecycle. The design is grounded in three recent systematization-of-knowledge papers on runtime integrity and trusted execution environments.

Index Terms—large language models, integrity attestation, trusted execution environment, TPM, Linux Integrity Measurement Architecture, runtime monitoring, security, threat modeling

I. INTRODUCTION

Large language models are increasingly deployed in production environments where they handle sensitive queries, generate code, and influence downstream decisions. Here, the model is usually considered a trusted module because it is initialized once from the storage media, then mapped into the address space of the process that uses it for serving inferencing requests during its lifetime. Verification of the deployed model’s integrity is not done after deployment, and the issue of whether the model running at the moment corresponds to the one which should have been deployed is an unspoken assumption.

This problem becomes especially critical when one considers that the machine learning model is nothing but data — weights, tokenizers, and configurations — and anyone who gains access to these files or to the loading process itself can alter the behavior of the model. An attacker can swap the weight file before load, modify the tokenizer to alter how

inputs are parsed, write directly to process memory after load through standard Linux interfaces, or manipulate inference-time parameters to push the model outside its authorized operating envelope. These attacks are well within the capabilities of an attacker who has compromised the system on which the model runs.

Existing defenses tend to address one stage of the model’s lifecycle. File-level integrity tools such as the Trusted Platform Module combined with the Linux Integrity Measurement Architecture (TPM+IMA) [4], [5] cover load-time tampering but stop tracking the model once its bytes are in process memory. Behavioral attestation approaches such as AttestLLM [6] cover runtime tampering by checking the live model against secret triggers held in a Trusted Execution Environment (TEE), but they do not provide load-time guarantees. The literature treats these as alternative approaches; in practice, neither alone covers the full attack surface against a deployed LLM.

This paper presents LLMGuard, an integrity attestation framework that combines the two approaches into a layered design. The framework targets an adversary model in which the LLM process is itself the attacker: the prover and the adversary are the same principal. Under this model, the framework provides hardware-rooted load-time file integrity via TPM+IMA, and TEE-protected runtime behavioral attestation in the AttestLLM style, with the two layers covering complementary points in the model’s lifecycle. The design is grounded in three recent systematization-of-knowledge papers on runtime integrity and trusted execution environments [1]–[3].

The contributions of this work are:

- A threat model for integrity attacks on locally deployed language models, defining five attack categories on the model’s lifecycle and a precise adversary model in which the prover is fully attacker-controlled.
- A comparative analysis of five attestation approaches from the literature against this threat model, identifying that no single approach covers the in-scope attack surface alone.
- A layered design that combines TPM+IMA and TEE-based behavioral attestation, with an explicit argument for why the combination is necessary, grounded in the time-of-check-to-time-of-use gap that file-integrity approaches

leave open.

- An honest scoping of the design’s limitations, including the strengths and weaknesses of hardware and software TEE implementations and a discussion of which adversary classes the design does not address.

The remainder of the paper is organized as follows. Section II provides background on attestation, TPM-based measurement, and TEEs. Section III defines the threat model. Section IV positions the work in the broader literature. Section V surveys the candidate attestation approaches and presents the chosen design. Section VI describes the architecture. Section VII discusses limitations, Section VIII outlines future work, and Section IX concludes.

II. BACKGROUND

This section provides background on the concepts the paper builds on: remote attestation, TPM-based measurement, and trusted execution environments. Readers familiar with these concepts may skip to Section III.

A. Remote Attestation

Remote attestation is a class of protocol in which one party — the *verifier* — obtains evidence that another party — the *prover* — is in a known-good state, where “state” is typically defined in terms of the software the prover is executing and the data the prover holds. The evidence is generated by the prover but authenticated by a trust anchor that is known to both the prover and the verifier, ensuring that the prover cannot create false evidence. A verifier that receives valid attestation evidence concludes that the prover is in the expected state and proceeds to trust the prover’s outputs; a verifier that receives invalid or no evidence refuses to trust the prover.

The literature distinguishes *attestation* from *auditing* [1]. In attestation, the verifier requests evidence and refuses to trust the prover’s outputs if the evidence is missing or invalid; the protocol is passive on the verifier side. In auditing, the system is required to deliver evidence to the verifier even when the prover is compromised, so that root-cause analysis is possible after a failure. LLMGuard, as presented in this paper, follows the attestation model.

B. TPM-Based Integrity Measurement

The *Trusted Platform Module* (TPM) [5] is a tamper-resistant hardware component, typically a discrete chip or firmware component on the system board, that provides a small set of cryptographic primitives the rest of the system can rely on. The TPM holds a set of *Platform Configuration Registers* (PCRs), each of which is a fixed-size register that can be *extended* but not arbitrarily written: the only way to change a PCR’s value is to provide new data, which the TPM hashes together with the current PCR value to produce the new value. A PCR therefore accumulates a cryptographic summary of every value ever extended into it since the last hardware reset.

The *Linux Integrity Measurement Architecture* (IMA) [4] is a kernel subsystem that uses the TPM to record measurements

of files as they are loaded into memory. When the kernel handles a file load — through `mmap`, `open`, or `execve` — IMA computes a SHA-256 hash of the file’s contents, extends the hash into a designated PCR, and appends an entry to an in-kernel measurement log. A verifier that later requests a TPM-signed quote of the PCR value, together with the measurement log, can replay the log and confirm both the integrity and the order of every file the system has loaded since boot.

C. Trusted Execution Environments

A *Trusted Execution Environment* (TEE) is an isolated execution context, separate from the normal operating system, in which code and data are protected from inspection or modification by other software on the same machine. A TEE provides four key properties [2]. *Verifiable launch* allows an external entity to verify that the TEE started execution with the correct code. *Runtime isolation* prevents any other software from accessing the TEE’s memory. *Trusted I/O* enables communication with peripherals that bypasses untrusted software. *Secure storage* provides data persistence so that secrets can be retrieved only by the corresponding TEE.

TEEs are implemented in several ways. *Hardware-supported* TEEs use CPU features to enforce isolation: Intel SGX uses processor-enforced enclaves, AMD SEV-SNP uses encrypted virtual machines, and ARM TrustZone uses a secure-world execution mode. *Software-supported* TEEs use a trusted hypervisor or microkernel to enforce isolation between domains running on the same hardware, without dedicated CPU features for isolation. Examples include hypervisor-based isolation domains and microkernel-based constructions such as Komodo and SANCTUARY [2]. The choice between hardware and software implementations is a deployment decision; the security properties they provide differ in strength but the design abstractions they expose are similar.

D. Behavioral Attestation for Language Models

Recent work has extended attestation from file-level integrity to model behavior. AttestLLM [6] embeds a watermark in a language model by fine-tuning it so that secret trigger inputs produce specific expected activation patterns at certain internal layers. The triggers and expected activations are stored inside a TEE. At runtime, the TEE periodically sends a trigger through the running model, observes the activation pattern, and compares it to the expected value. A tampered model — whether tampered on disk before load, in memory after load, or through manipulated inference parameters — produces a different activation pattern and the check fails. Behavioral attestation thus provides a runtime integrity guarantee that complements file-level integrity by verifying what the model is actually computing, not just what its files look like.

III. THREAT MODEL

A. Roles

The design distinguishes three roles based on privilege level and what each is trusted to do. The *root* is the system administrator role corresponding to `UID 0` on the Linux

system. It configures the platform at install time, provisioning the TPM, enrolling baseline measurements, and registering the verifier daemon under systemd. Root is the ultimate trust anchor; if root is compromised, the entire system’s integrity is void. The *verifier user* is a dedicated Linux account that runs the LLMGuard verifier daemon. The verifier user account owns three files that no other account can read or modify: the baseline, the signing key, and the append-only audit log. The verifier user is distinct from the account that runs the language model. The *attacker* is the Linux account under which the model process executes. The attacker fully owns the model process and can do anything a process owner can do within the bounds of the kernel. This includes modifying any file the account owns and reading or writing the memory of its own process. The attacker cannot become root or the verifier user.

B. Adversary Capabilities

The attacker has full ownership of the LLM process and the user account it runs under. With that, the attacker can modify any file owned by that account, including the model weights, tokenizer files, and configuration. The attacker can read and write the model process’s own memory through standard Linux interfaces such as `/proc/self/mem` (which provides direct access to the process’s virtual address space) and `ptrace` (which provides observation and control of the process’s execution). The attacker can use `LD_PRELOAD` to load arbitrary shared libraries into its own processes and monkey-patch the Python interpreter or any loaded module at runtime. The attacker controls all input and output the model process performs and chooses when, whether, and how to respond to requests from the verifier. In short, the attacker has every capability that a Linux user normally has over their own processes and files.

C. What the Adversary Cannot Do

The attacker is bounded by the operating system kernel and the TPM. The attacker cannot escalate to root or otherwise gain kernel-level privileges, whether by exploiting kernel vulnerabilities or by loading malicious drivers. The attacker cannot read or write files owned by the verifier user, cannot read or write the verifier daemon’s process memory, and cannot forge a signature without the verifier’s private key. The attacker cannot modify kernel-managed state, including the loaded kernel image and the IMA measurements stored in TPM Platform Configuration Registers (PCRs), and cannot reset PCRs except by hardware power cycle. Physical attacks on the machine are out of scope.

All of these limitations are enforced by the kernel and the TPM hardware, and the integrity of the system depends on the correctness of these two components. Any kernel exploit, TPM compromise, or physical access to the machine invalidates the guarantees of the security architecture proposed. Section VII discusses these out-of-scope adversaries in more detail.

D. Attacks in Scope

LLMGuard targets five categories of attack on the locally deployed language model.

- 1) **A1: At-rest tampering.** The attacker modifies or replaces the model weight files on disk before they are loaded by the inference process. Since the model file lives in attacker-owned storage, the attacker has direct write access and needs no special privileges to tamper with it. The intent is to have the modified files mapped into the model process at load time, so that any subsequent inference operates on the tampered file.
- 2) **A2: Tokenizer tampering.** The attacker modifies the tokenizer file on disk to alter the model’s interpretation of inputs without touching the model weights themselves. This attack is distinct from A1 because the two target different assets and an attacker may choose to tamper with one and not the other.
- 3) **A3: In-memory tampering during runtime.** After the model has been loaded into memory, the attacker modifies the loaded model directly. The on-disk files remain unchanged, so file-level integrity checks alone do not detect this attack.
- 4) **A4: Runtime parameter manipulation.** The attacker alters inference-time parameters such as temperature and top-*p* with the aim of having the model produce outputs that are outside its authorized policy. This may mean neutralizing a safety-oriented system prompt, setting temperature high enough to produce chaotic output, or removing output length limits. This attack happens entirely at the inference call site by passing unauthorized arguments.
- 5) **A5: Execution-state tampering.** The attacker modifies the model’s internal computational state during inference in order to manipulate output without modifying the model weights or input parameters. This attack is acknowledged but not addressed by the present design; it is deferred to future work.

A1 and A2 target the file-system layer, where files exist before they are mapped into process memory. A3 and A4 target in-memory state during execution, where file-level defenses no longer apply. A5 targets the execution path itself. No single attestation primitive in the literature covers A1 through A4 simultaneously, which motivates the layered design described in Section V.

E. Attacks Out of Scope

Several attack categories are explicitly excluded from the present threat model and are not addressed by LLMGuard.

Kernel-level attacks. An attacker who has exploited a vulnerability in the Linux kernel, loaded a malicious kernel module, or otherwise gained kernel-level privileges defeats every guarantee the design makes. The kernel mediates IMA measurement, enforces verifier-prover isolation, and brokers all access to the TPM. Defending against kernel-level adversaries requires hardware-rooted isolation of the verifier itself and is orthogonal to this work.

Side-channel and microarchitectural attacks. Cache-timing attacks, controlled-channel attacks against page-table-mediated isolation, speculative-execution attacks, and

ciphertext side channels are out of scope. This scoping is consistent with the threat models declared by commercial TEE deployments [3].

Physical attacks. Cold-boot attacks, bus snooping, voltage-based fault injection, and direct hardware tampering with the TPM require physical access to the machine and are out of scope.

Supply-chain attacks on the model. If the model arrives already backdoored from upstream, the problem becomes one of model provenance rather than runtime integrity. Provenance is addressed by separate research on supply-chain security for ML pipelines and is not within the scope of this work.

Network-level adversaries. Tampering with attestation reports in transit is assumed to be addressed by transport-layer security at the integration boundary and is orthogonal to this work.

IV. RELATED WORK

This section positions LLMGuard within three recent systematization papers on runtime integrity and trusted execution environments. The papers are not surveyed in detail; each is summarized briefly and its relevance to the present work is noted.

A. Systematization of Runtime Integrity

Ammar et al. [1] systematize runtime integrity defenses, framing the literature around two complementary protections: control-flow integrity (CFI), which prevents control transfers outside an expected control-flow graph, and control-flow attestation (CFA), which records control transfers and produces tamper-proof evidence of them for a remote verifier. CFI is a local, prevention-oriented mechanism; CFA is an evidence-producing, attestation-oriented mechanism. The authors argue that runtime integrity is the “last line of defense” after prevention has failed.

The principles behind CFI and CFA do not apply to language models, which are dominated by matrix multiplication operations, not control flow operations. But there are two insights from this paper that are immediately applicable. First, the distinction between attestation and auditing — whether evidence is requested by the verifier or proactively delivered by the prover — frames LLMGuard’s evidence-production model (Section II). Second, the formalization of the time-of-check-to-time-of-use (TOCTOU) gap in remote attestation [1] provides the conceptual foundation for the layered design argument in Section V-C: load-time measurement alone is insufficient because the state at the time of use may differ from the state at the time of measurement.

B. Systematization of Trusted Execution Environments

Schneider et al. [2] systematize hardware-supported TEEs across commercial and academic designs, including Intel SGX, AMD SEV-SNP, ARM TrustZone, ARM Confidential Compute Architecture, IBM Protected Execution Facility, and academic RISC-V proposals such as Keystone, Sanctum, Penglai, and CURE. The authors decompose TEE designs

into four security goals — verifiable launch, runtime isolation, trusted I/O, and secure storage — and tabulate which designs provide each. They distinguish between Static Root of Trust for Measurement (SRTM), used by AMD SEV-SNP and ARM Confidential Compute Architecture, and Dynamic Root of Trust for Measurement (DRTM), used by Intel SGX and TDX.

In terms of the current research, the four-objectives framework serves as the perspective for analyzing the runtime layer’s TEE needs (Section VI). LLMGuard’s TPM+IMA layer uses the SRTM approach. In addition, the authors recognize the existence of software-based implementations of TEEs that can act as substitutes to hardware-based TEEs, mentioning Komodo and SANCTUARY as examples.

C. Limitations of Trusted Execution Environments

Li et al. [3] catalog known attacks against deployed TEEs. The attacks span six categories: vulnerable system designs (such as register leakage in early AMD SEV), controlled-channel attacks against page-table-mediated isolation, microarchitectural side channels (cache, branch predictor, performance counter), speculative-execution attacks (Foreshadow, ZombieLoad, RIDL, CacheOut), microarchitectural flaws (ciphertext attacks against AMD SEV’s memory encryption), and hardware fault attacks (Plundervolt, VoltPillager, Rowhammer-induced TEE lockdown). The authors note that side-channel attacks are explicitly excluded from the threat models of commercial TEEs — a scoping decision that has been criticized but that reflects the practical limits of what hardware isolation can provide.

This paper is critical for understanding the limits of LLMGuard’s runtime layer. A hardware TEE provides strong isolation guarantees within its threat model, but the threat model excludes a long list of attack classes that have been demonstrated in practice. LLMGuard inherits these limitations when using a hardware TEE, and inherits a different set of limitations — specifically, vulnerability to kernel-level adversaries — when using a software TEE. The Limitations section (Section VII) discusses these tradeoffs in detail.

V. DESIGN OVERVIEW

A. The Decision Space

Section III defines five categories of attack on a deployed language model. Building a defense against this threat model requires choosing among existing attestation approaches in the literature, each of which makes different assumptions about hardware, trust, and the kind of evidence it produces. After surveying five candidate approaches, we found that none of them covers all in-scope attacks (A1 through A4) on its own, and that the approaches differ substantially in hardware requirements and practicality. Table I summarizes how each candidate maps to the threat model. The rest of this section walks through the candidates, the chosen design, and the rejected alternatives.

B. Candidate Approaches

1) TPM with the Linux Integrity Measurement Architecture:

The Integrity Measurement Architecture (IMA) [4] is a Linux kernel security subsystem that maintains a runtime measurement list of files (executables, libraries, configurations) and anchors an aggregate hash in a hardware Trusted Platform Module (TPM) [5] to prevent tampering. The kernel’s IMA module hooks into the file loading code paths (`mmap`, `execve`, and related syscalls). When any process asks the kernel to load a file into memory, IMA reads the file, computes a SHA-256 hash, extends the hash into a Platform Configuration Register (PCR) inside the TPM, and appends an entry to an in-kernel measurement log recording the file path, hash, and timestamp. PCRs are hardware registers inside the TPM that can only be extended, never written arbitrarily, so the current PCR value is a cryptographic summary of every measurement that has been extended into it since boot, in order.

The attacker cannot modify kernel code, cannot access the TPM’s signing key, and cannot alter PCRs through legitimate extension operations. The attacker therefore has no opportunity to intercept the hashing, because it happens in a privilege domain they cannot reach. The measurement log lives in kernel memory, which the attacker cannot directly read or write. As a result, the attacker cannot insert fake entries, remove entries corresponding to tampered files, or reorder entries.

This approach gives full coverage for A1 and A2. The measurement log records the hash of whatever file was actually loaded; if the file is backdoored, the recorded hash will not match the baseline hash, and the attestation will reject the file. However, once the model file has been loaded into the attacker’s process memory, the attacker has full write access and can modify in-memory weights as well as in-memory values produced during inference. This approach therefore does not protect against A3 or A5. Additionally, runtime inference parameters are not files, and IMA does not hook function arguments, so there is no protection against A4.

IMA also does not protect against the attacker choosing to skip verification entirely. The attacker could request a TPM quote, observe that it reports the correct state, and then tamper with memory before running inference. The attestation itself is trustworthy, but a one-time attestation does not guarantee anything about subsequent state. This is addressable by continuous attestation, which IMA is not designed for, and which motivates the runtime layer described below.

2) TEE-Based Attestation with Activation Watermarking:

A Trusted Execution Environment (TEE) is a secure, isolated area within a main processor (CPU) that protects sensitive code and data from the rest of the system, including the operating system. The TEE can produce a signed measurement of itself stating what code is loaded inside and what initial data was provisioned. AttestLLM [6] adds an LLM-specific layer on top of this hardware attestation. LLMs are too large to fit entirely inside a typical TEE, and running the full model outside the TEE while keeping something useful inside requires a cryptographic binding between the model and the TEE.

In this paper, we treat TEE as a category that includes both hardware-supported implementations (such as Intel SGX, AMD SEV-SNP, and ARM TrustZone) and software-supported implementations (such as hypervisor-based isolation domains). The choice of TEE implementation is a deployment-time decision; the design depends on the properties of a TEE — runtime isolation and secure storage — rather than on any specific implementation.

AttestLLM uses activation-based watermarking. At setup, the model is fine-tuned so that specific secret trigger inputs produce specific expected activation patterns at certain internal layers. The triggers and expected activations are secrets stored inside the TEE. The model itself runs in process memory. The TEE periodically sends trigger inputs through the model, observes the resulting activations, and verifies that they match the expected patterns. If the model has been tampered with, the activations diverge and the watermark check fails, raising an alert. The attacker cannot predict in advance which trigger inputs will be used, so cannot prepare a tampered model that responds correctly to those specific inputs. The attacker also cannot see the expected activation patterns, so cannot reverse-engineer the watermark scheme from outside the TEE.

The TEE’s own attestation also covers the verification code itself. A downstream consumer of an attestation report learns not just that “the model claims to have the watermark” but that “a genuine TEE containing the correct verification logic confirmed the watermark was present.” This is a stronger guarantee than file-level attestation alone.

If the attacker modifies the model file in any way, the loaded model will no longer contain the watermark, and the activations will not match the expected pattern. This protects against A1, although compared with the TPM+IMA approach, this approach catches A1 during runtime rather than at load time. The tokenizer, on the other hand, is not directly watermarked, so A2 is not protected against: a tampered tokenizer could alter inputs in a way that still produces plausible activations. The approach is strongest against A3, where in-memory weight tampering produces different activations for the trigger inputs and the watermark check catches the discrepancy. In a deployment where the TEE itself is the entity that serves inference requests, A4 is also protected at the TEE boundary. A5 remains an open area; ongoing inference would detect some forms of activation tampering, but AttestLLM does not claim full execution-state coverage.

3) *Runtime Segmented Attestation:* An LLM with several gigabytes of weights cannot be hashed every second without destroying inference performance, but infrequent hashing leaves long windows in which an attacker can tamper with memory and escape detection. In runtime segmented attestation, the model is divided into many small segments, each a few megabytes. On each attestation round, a random subset of segments is selected and hashed. The subset is small enough that hashing completes quickly. The selection is randomized and not known to the attacker in advance. At a high frequency of attestation rounds, the attacker cannot predict which segments will be checked next. No specific

hardware is required for this approach; any trusted location can run the attestation logic.

If the attestation code runs inside the attacker’s own process, the approach fails completely, as the attacker can rewrite it. If the attestation code runs under a different Linux user, it cannot read the attacker’s process memory without root privileges or specific capabilities. This is the practical constraint: attestation logic needs to be both outside the attacker’s reach and able to observe the attacker’s memory. The cleanest solution is to run the attestation logic inside a TEE with hardware-enforced access to the memory it chooses to monitor.

Coverage against the threat model is concentrated on A3. With high-frequency segment sampling, in-memory weight tampering is caught probabilistically: an attacker who modifies even a small fraction of weights eventually intersects with the randomly chosen segment set. The approach gives partial coverage of A1 because a fully replaced model would also produce different hashes, but the detection mechanism is slower and less efficient than dedicated load-time measurement. Tokenizer files and inference-time parameters (A2 and A4) are outside the scope of segment-level memory hashing, and execution-state tampering (A5) is not addressed. The strength of this approach is detecting in-memory tampering without requiring file-level integrity infrastructure; its weakness is the requirement for either privileged memory access or TEE-hosted attestation logic to remain trustworthy under the threat model.

4) *Software-Based Attestation*: Software-based attestation, exemplified by SWATT [8] and the family of memory-printing protocols, attempts to detect tampering on a remote device without relying on any trusted hardware. The verifier sends a cryptographic challenge to the prover. The prover then runs a carefully designed function that traverses its entire memory in a sequence determined by the challenge, accumulating a checksum as it goes. The verifier knows the memory contents the prover is supposed to have and computes the same checksum locally with a known reference time bound.

The trust assumption is timing. The function is designed so that any deviation from the expected memory traversal — such as redirecting reads to a hidden clean copy or substituting precomputed values — introduces detectable delay. If the prover’s response arrives within the expected time window with the correct checksum, the verifier concludes that the prover ran the function on the actual memory contents. If the response is too slow, or the checksum is wrong, tampering is presumed.

Several aspects of this approach do not fit our threat model. The technique was designed for embedded devices with predictable timing characteristics, a single trusted memory configuration, and an attacker who is hiding malicious code in memory. Our setting is the opposite on each dimension: a server-class system with cache hierarchies and unpredictable timing, a multi-process Linux environment in which the attacker is the legitimate user of the process, and an attacker who has no need to hide because they own the process outright. Additionally, the technique requires that the verifier and the

prover share a precise model of memory contents, which is incompatible with the size and dynamic memory layout of modern LLMs.

Software-based attestation gives at best partial coverage of A1 through A3 in a forced adaptation, and no coverage of A4 or A5. We exclude it from the design.

5) *Zero-Knowledge Machine Learning*: Zero-knowledge machine learning (zkML) takes a different approach. Rather than attesting to the integrity of the model state, it produces a cryptographic proof that a specific output was computed by a specific model on a specific input, without revealing the model parameters themselves. The verifier checks the proof and is convinced that the computation was performed correctly. Recent work, notably zkLLM [9], has extended these techniques to large transformer-based models.

The coverage profile is theoretically complete. A valid zkML proof attests not just to file integrity or in-memory state but to the actual computation performed during inference. A1 through A5 are all covered: if the model file is tampered with, the proof fails; if weights in memory are modified, the proof fails; if inference parameters are manipulated, the proof reflects the manipulated values; if execution state is altered during a forward pass, the proof fails. There is no gap, because the proof is over the computation itself.

The limitation is practicality. Current zkML implementations require minutes to hours to generate a proof for a single token of LLM inference, even with substantial optimization, and the constant overhead grows with model size and proof complexity. While zkML may become viable as cryptographic machinery improves, it is not currently a practical option for real-time inference workloads. We acknowledge zkML as the theoretical ceiling for attestation coverage and as a possible long-term direction, but exclude it from the design on cost grounds.

TABLE I
COMPARISON OF ATTESTATION APPROACHES AGAINST THE THREAT MODEL

Approach	A1	A2	A3	A4	A5	HW	Used
TPM + IMA	✓	✓	×	×	×	TPM	✓
AttestLLM-style	✓	~	✓	~	~	TEE	✓
Segmented attest.	~	×	✓	×	×	TEE	×
SW-based attest.	~	~	~	×	×	—	×
zkML	✓	✓	✓	✓	✓	—	×

✓ = covered, ~ = partial, × = not covered

C. Chosen Design

Of the five candidates surveyed, LLMGuard combines two: TPM with the Linux Integrity Measurement Architecture for load-time integrity, and TEE-based behavioral attestation in the AttestLLM style for runtime integrity. Layer one targets A1 and A2 with hardware-rooted file measurement. Layer two targets A3 and partially targets A4 with continuous behavioral verification inside a TEE. A5 is acknowledged in Section III-D and deferred to future work; we discuss it again in Section VII.

The combination is necessary because each layer alone leaves a structural gap that only the other layer can close.

TPM+IMA measures files at load time and binds those measurements to hardware-rooted Platform Configuration Registers. Once the file’s bytes are loaded into process memory, however, IMA stops tracking them. An attacker who modifies the in-memory copy of the model through interfaces such as `/proc/self/mem` or `ptrace` produces a system whose running state has diverged from its measured state. This is a time-of-check to time-of-use (TOCTOU) gap, well-documented in the runtime attestation literature [1]. Closing this gap requires a mechanism that observes the live model rather than the on-disk file. The TEE-based behavioral attestation layer provides exactly this: it keeps secret trigger inputs and expected activation patterns inside the TEE, periodically exercises the running model with those triggers, and detects in-memory tampering as a divergence between observed and expected activations. Conversely, behavioral attestation alone does not provide load-time guarantees: an attacker who tampers before the verifier’s first runtime check has a window of unmonitored exposure, which TPM+IMA closes with its hardware-rooted load-time measurement. The two layers operate at different points in the model’s lifecycle, each defending the points the other cannot reach.

D. Rejected Alternatives

The three remaining candidates were considered but excluded from the design.

Runtime segmented attestation [7] was evaluated as a software-only alternative to AttestLLM for the runtime layer. It provides probabilistic detection of in-memory tampering through randomized periodic hashing, but lacks the cryptographic strength of TEE-based attestation against an attacker with full process ownership. Additionally, the approach requires either privileged memory access or TEE-hosted attestation logic to remain trustworthy under our threat model, which reintroduces a TEE dependency without the corresponding behavioral verification benefits. We treat segmented attestation as a viable software-only fallback for deployments without TEE hardware but do not select it for the primary design.

Software-based attestation was excluded because its threat model does not match this work. The technique was designed for embedded devices with active malware in hiding; our setting is a multi-process server where the attacker is the legitimate user of the process, not a hidden adversary. The technique’s timing assumptions do not hold on server-class hardware, and its memory model is incompatible with the size of modern LLMs.

Zero-knowledge ML was excluded on practical rather than theoretical grounds. While zkML covers all five attack categories in principle, current proof generation rates for billion-parameter models are on the order of minutes per token [9], rendering the approach infeasible for real-time inference. We expect zkML to become viable as the underlying cryptographic machinery matures; for now, hardware-rooted and TEE-rooted approaches are the only practical options.

VI. ARCHITECTURE

A. Trust Domains

LLMGuard runs on a single Linux machine and partitions the system into three trust domains. The *trusted foundation* consists of the Linux kernel and the TPM. The kernel enforces process isolation and runs the IMA module that performs load-time measurement. The TPM is a hardware chip that stores integrity measurements in Platform Configuration Registers (PCRs) that no userspace process can tamper with. The *verifier domain* is a dedicated Linux user account that runs the LLMGuard verifier daemon. It owns the baseline measurements, the verifier’s signing key, and an append-only audit log. The *prover domain* is a separate Linux user account under which the LLM process executes. Under our threat model (Section III), the prover is fully attacker-controlled.

The kernel enforces the separation between the verifier domain and the prover domain through standard Linux user isolation: distinct UIDs, file ownership and permission enforcement, and the inability of one user to read or write another user’s process memory. Root configures this arrangement at install time — creating the user accounts, setting file ownership, provisioning the TPM, and registering the verifier daemon with `systemd` — and is otherwise passive at runtime.

B. Components

In the verifier space, there are four logical pieces that work together to generate attestations. The first is the *baseline store*, which has the hash values that the verifier looks for at startup. These baselines are recorded once, at install time, by a trusted party. The *signing key* is a private key held only by the verifier user; attestation reports produced by the verifier are signed with this key so that downstream consumers can confirm the report came from the legitimate verifier. The *audit log* is an append-only record of every attestation round, its inputs, and its outcome. The *policy engine* compares observed measurements and behavioral check results against the baseline and decides whether the prover is in a known-good state.

Within the prover domain, the LLM process runs normally: it loads the model file, the tokenizer, and the configuration from disk, and serves inference requests. The process is fully attacker-controlled and we make no assumptions about what it does internally. The integrity of the prover is not guaranteed through any inspection of the prover but rather through evidence gathered about the prover externally: TPM quote from the kernel measurement chain and behavioral checks from the TEE.

The TEE itself sits outside both userspace domains. In a hardware-supported deployment, it is provided by the CPU’s enclave or virtualization features (Intel SGX, AMD SEV-SNP, ARM TrustZone). In a software-supported deployment, it can be provided by a hypervisor-based isolation domain or a separate virtual machine running under a trusted hypervisor. The TEE holds the secret trigger inputs and the expected activation patterns required by the runtime layer, and it is

the component that exercises the prover with triggers and compares the results to the expected values.

C. Attestation Flow

A single round of attestation proceeds in three stages.

First, *load-time measurement*. When the LLM process invokes a file load — through `mmap`, `open`, or `execve` — the kernel’s IMA module computes a SHA-256 hash of the file, extends the hash into a designated PCR inside the TPM, and appends an entry to the in-kernel measurement log. The prover cannot interfere with this step because IMA executes in kernel space and the PCR is in tamper-proof hardware.

Second, *evidence collection*. The verifier daemon requests a signed quote of the current PCR value from the TPM, bound to a fresh verifier-generated nonce. The TPM signs the quote with its hardware-resident attestation key. The verifier also requests the most recent behavioral check result from the TEE, which contains the activation pattern observed for the most recent trigger and the expected pattern.

Third, *evaluation*. The verifier replays the measurement log against the signed PCR value to confirm internal consistency, compares the measurements to the baseline, and confirms that the latest behavioral check matched the expected activation pattern. If all checks pass, the verifier appends a positive attestation record to the audit log and signs it. If any check fails, the verifier records the failure and the system can be configured to alert, halt the LLM process, or trigger an out-of-band response.

This flow runs continuously. The runtime layer’s behavioral checks execute on a regular schedule independent of verifier requests, so that an attacker who never triggers a verifier query still cannot avoid detection: the TEE will eventually fail a behavioral check, and the next attestation round will surface the failure.

D. What the Attacker Sees

The system appears to be just another regular Linux machine executing the LLM process from the viewpoint of the attacker. The attacker is able to manipulate the model files, access the memory space of the process, and even interfere with library function calls. What the attacker cannot do is reach into the verifier domain, modify the kernel, tamper with the TPM, or read the secrets inside the TEE. The kernel and the TPM are hardware-or-firmware-enforced boundaries. The TEE’s isolation depends on its implementation: a hardware TEE protects against userspace and (depending on the TEE) kernel-level adversaries; a software TEE protects against userspace adversaries only. Section VII discusses these trade-offs.

VII. LIMITATIONS

In the following, we describe four limitations that apply to LLMGuard according to its current design. The first two have been taken directly from our threat model scope. The third limitation is related to the TEE design decision. The fourth is the absence of empirical validation.

A. Out-of-Scope Adversaries

LLMGuard does not defend against several classes of adversary that fall outside the threat model in Section III. A kernel-level attacker — one who has successfully exploited a vulnerability in the Linux kernel or installed a malicious kernel module — defeats every guarantee the design makes. The kernel mediates the IMA measurement chain, enforces the verifier-prover isolation, and brokers all access to the TPM. A compromised kernel can fabricate measurements, read verifier memory, and produce signed evidence indistinguishable from a legitimate verifier’s. Side-channel attacks — cache-timing attacks, controlled-channel attacks, power-analysis attacks — are also out of scope. We note that this scoping is consistent with how commercial TEE deployments scope their threat models [3]. Physical attacks, including cold-boot attacks, bus snooping, and direct hardware tampering with the TPM, require physical access to the machine and are similarly out of scope. Network-level adversaries are assumed to be addressed by transport-layer security at the integration boundary and are orthogonal to this work.

B. A5 Is Not Addressed

Execution-state tampering (attack A5 in Section III-D) is identified in the threat model but not addressed by the present design. Detecting modifications to logits, attention values, or KV-cache contents during a forward pass would require techniques substantially beyond the scope of TPM+IMA file integrity or AttestLLM-style behavioral attestation. A zkML-based approach (Section V) covers A5 in principle but is not currently practical at LLM scale [9]. We defer A5 as future work.

C. TEE Implementation Tradeoffs

The strength of LLMGuard’s runtime layer depends on the strength of its TEE. As discussed in Section VI, the TEE can be implemented in hardware (Intel SGX, AMD SEV-SNP, ARM TrustZone) or in software (a hypervisor-based isolation domain). The two choices yield different guarantees.

A hardware TEE provides isolation against userspace and, depending on the TEE, kernel-level adversaries. This is the strongest available implementation, and a deployment of LLMGuard that uses a hardware TEE provides the strongest guarantees the design can offer. However, hardware TEEs themselves have well-documented limitations. Cache-timing side channels, controlled-channel attacks against page-table-mediated isolation, speculative-execution attacks such as Fore-shadow [10] and ZombieLoad [11], ciphertext side channels against AMD SEV’s memory encryption, and hardware-level fault attacks such as Plundervolt [12] and VoltPillager [13] have all been demonstrated against deployed hardware TEEs [3]. Hardware TEE selection is thus not an option that guarantees absolute security but is rather one for greater isolation from a certain category of adversaries.

A software TEE — such as a separate virtual machine running under a trusted hypervisor, or a microkernel-based isolation domain like Komodo or SANCTUARY — provides

isolation against userspace adversaries through hypervisor-enforced memory and execution separation. This is sufficient against the threat model defined in Section III, in which the attacker is constrained to userspace privileges of the LLM process. A software TEE does not, however, defend against a kernel-level attacker who has compromised the hypervisor itself, nor against the side-channel and microarchitectural attacks that affect hardware TEEs and any code running on the same physical hardware.

In neither case does LLMGuard claim defense against an attacker who has fully compromised the trusted computing base on which the TEE depends. The design trades off strength against deployability: a hardware TEE provides stronger guarantees where the hardware is available, and a software TEE provides workable guarantees on commodity systems at lower assurance.

D. No Empirical Validation

This work is a design contribution. Detection rates, false-positive rates, attestation latency, the performance overhead of continuous behavioral checks, and the system’s behavior under load have not been measured. Future work will require an implementation phase and an evaluation phase to establish these properties empirically.

VIII. FUTURE WORK

Several directions follow from the present design.

Implementation and evaluation. The most direct extension is to implement the verifier daemon, the IMA integration, and the TEE-resident behavioral attestation component, and then to evaluate the resulting system. The evaluation should measure attestation latency, behavioral-check periodicity tradeoffs, false-positive rates against benign in-memory perturbations (such as autograd state changes or memory defragmentation), and overhead on inference throughput.

Strengthening the software-TEE construction. The software-TEE deployment described in Section VI relies on a trusted hypervisor for isolation. Future work could explore stronger software constructions, including microkernel-based isolation domains in the style of Komodo or SANCTUARY [2], and confidential-VM implementations that minimize the trusted-hypervisor surface.

A5 coverage. Detecting execution-state tampering during inference remains open. Two directions are worth exploring: deeper TEE-resident instrumentation that captures activation patterns at multiple layers per inference, and zkML approaches once cryptographic machinery permits feasible per-token proofs at LLM scale.

Continuous attestation and auditing. The current design produces attestation evidence on request. A stronger model is auditing [1], in which the verifier proactively pushes signed evidence to an external logging service on a schedule, and non-responsiveness is itself treated as a fault. This requires hardening the verifier-side scheduling against userspace interference — a kernel timer or watchdog — and is a natural extension of the existing design.

Multi-model and multi-tenant deployments. The present design assumes a single LLM process under a single attacker user. A natural extension is to handle multiple model processes on the same machine, each potentially under a different user, with a shared verifier and shared TPM but per-prover behavioral attestation. The TEE’s secret trigger sets and expected activations would need to be partitioned per model.

IX. CONCLUSION

This work presents LLMGuard, an integrity attestation framework for locally deployed large language models. The threat model addresses a userspace adversary with full control over the LLM process, defining five attack categories (A1–A5) on the model’s lifecycle. The design combines two attestation primitives from the literature: TPM-rooted load-time file integrity using the Linux Integrity Measurement Architecture, and TEE-protected runtime behavioral attestation in the AttestLLM style. Neither layer alone covers the in-scope attack surface; together, they close a time-of-check-to-time-of-use gap that is structural to file-level integrity approaches. The framework is implementable on commodity Linux hardware, with the TEE provided by hardware where available and by hypervisor-based software isolation otherwise.

The contributions of this work are the threat model, the comparative analysis of five candidate approaches, and the layered architecture. Implementation, empirical evaluation, and stronger constructions for the runtime layer are left as future work. The design is grounded in three recent systematization papers and is intended as a step toward deployable integrity guarantees for production LLM systems.

ACKNOWLEDGMENT

The author would like to acknowledge the contribution made by Dr. Eugene Vasserman from the Department of Computer Science at Kansas State University in providing advice for this project as well as the selection of literature for the same. The author also acknowledges the use of AI-assisted tooling (Anthropic Claude and OpenAI ChatGPT) for diagram generation and editorial assistance during the preparation of this manuscript; all design decisions, technical content, and final wording are the author’s own.

REFERENCES

- [1] M. Ammar, A. Caulfield, and I. De Oliveira Nunes, “SoK: Integrity, attestation, and auditing of program execution,” in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025.
- [2] M. Schneider, R. J. Masti, S. Shinde, S. Capkun, and R. Perez, “SoK: Hardware-supported trusted execution environments,” arXiv preprint arXiv:2205.12742, 2022.
- [3] M. Li, Y. Yang, G. Chen, M. Yan, and Y. Zhang, “SoK: Understanding design choices and pitfalls of trusted execution environments,” in *Proc. 19th ACM Asia Conf. on Computer and Communications Security (ASIA CCS ’24)*, 2024.
- [4] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and implementation of a TCG-based integrity measurement architecture,” in *USENIX Security Symposium*, 2004.
- [5] Trusted Computing Group, “Trusted Platform Module (TPM) summary,” 2008. [Online]. Available: <https://trustedcomputinggroup.org/resource/trusted-platform-module-tpm-summary/>

- [6] R. Zhang, Y. Zhao, N. Javidnia, M. Zheng, F. Koushanfar, "AttestLLM: Efficient attestation framework for billion-scale on-device LLMs," in *Proc. 63rd ACM/IEEE Design Automation Conference (DAC '26)*, 2026. arXiv:2509.06326.
- [7] S. Sen et al., "PracAttest: Practical attestation for edge devices running compute-heavy machine learning applications," in *ACSAC*, 2021.
- [8] M. Jakobsson and K.-A. Johansson, "Practical and secure software-based attestation," in *Workshop on Lightweight Security & Privacy: Devices, Protocols, and Applications (LightSec)*, 2011.
- [9] H. Sun and J. Zhang, "zkLLM: Zero-knowledge proofs for large language models," in *ACM CCS*, 2024.
- [10] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *USENIX Security Symposium*, 2018.
- [11] M. Schwarz, M. Lipp, D. Moghimi, J. Van Bulck, J. Stecklina, T. Prescher, and D. Gruss, "ZombieLoad: Cross-privilege-boundary data sampling," in *ACM CCS*, 2019.
- [12] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, and F. Piessens, "Plundervolt: Software-based fault injection attacks against Intel SGX," in *IEEE Symposium on Security and Privacy*, 2020.
- [13] Z. Chen, G. Vasilakis, K. Murdock, E. Dean, D. Oswald, and F. D. Garcia, "VoltPillager: Hardware-based fault injection attacks against Intel SGX enclaves using the SVID voltage scaling interface," in *USENIX Security Symposium*, 2021.